Roadmap proiect SSC - P4

Vectorizarea si optimizarea algoritmilor folosind instructiuni SIMD

- Student: Stefan Suciu, grupa 30236
- Disciplina: Structura Sistemelor de Calcul - componenta de proiect
- Categoria: proiect pentru nota maxima 8, cu imbunatatire originala pentru nota 10
- Platforma: PC x86 - AMD Ryzen 7 8845HS (Zen 4), Ubuntu sub WSL2, gcc
- Limbaj: C (C11), intrinsics SSE / AVX2 / AVX-512

---

1. Obiectivul proiectului

Implementarea a trei algoritmi cu operatii repetitive asupra vectorilor si matricelor, fiecare in doua variante:

- o varianta **scalara** (o operatie la un moment dat), folosita ca referinta;
- o varianta **vectorizata explicit prin intrinsics** (mai multe operatii in paralel, cu o singura instructiune).

Pentru fiecare algoritm se masoara timpul de executie si speedup-ul fata de varianta scalara. Se analizeaza efectul alinierii datelor, al dimensiunii vectorilor (128 / 256 / 512 biti) si al modului de acces la memorie.

2. Algoritmii alesi

Nr.	Algoritm	Tip date	Ce se urmareste
A1	Produs scalar a doi vectori	float	reducerea (suma orizontala), FMA
A2	Inmultirea a doua matrice NxN	float	accesul la memorie (randuri vs coloane), cache
A3	Convolutie 2D 3x3 pe imagine grayscale	uint8	procesare de pixeli, tratarea marginilor

3. Imbunatatirea propusa pentru nota 10

Optimizarea folosirii procesorului si a memoriei cache, peste vectorizarea de baza:

1. **Acumulatori multipli si desfasurarea buclelor (loop unrolling)** la produsul scalar si la inmultirea matricelor. In loc de un singur registru in care se aduna rezultatul, se folosesc 4 registre independente. Procesorul poate executa astfel mai multe instructiuni FMA in paralel, pentru ca nu mai asteapta rezultatul celei anterioare.
2. **Cache blocking (tiling)** la inmultirea matricelor. Matricele sunt impartite in blocuri mici care incap in cache-ul L1/L2, astfel incat datele sunt refolosite cat timp sunt inca in cache, in loc sa fie aduse de mai multe ori din RAM.

3. **Extinderea la AVX-512** si comparatia SSE (128 biti) vs AVX2 (256 biti) vs AVX-512 (512 biti) pe acelasi procesor, cu interpretarea rezultatelor (pe Zen 4, AVX-512 este executat intern ca doua operatii de 256 de biti).

Beneficiul demonstrabil: speedup suplimentar fata de varianta vectorizata simpla, masurat si prezentat in grafice. Ambele tehnici sunt implementate doar cu cod C simplu, fara biblioteci externe.

> Imbunatatirea trebuie discutata si validata cu cadrul didactic (laboratorul 3).

---

4. Planificarea pe laboratoare

Laboratorul de proiect are loc o data la doua saptamani. Proiectul trebuie finalizat pana la **laboratorul 6**.

Laboratorul 3 - Organizare si fundatie

De predat la laborator:

- [ ] Repository privat pe GitHub, cu acces pentru cadrul didactic
- [ ] `ROADMAP.md` (acest document)
- [ ] `CADRU_CONCEPTUAL.md` (documentul de context)
- [ ] Validarea cu profesorul a algoritmilor alesi si a imbunatatirii pentru nota 10

De facut pana la laboratorul 4:

- [ ] Instalarea mediului: WSL2 + Ubuntu, gcc, make, git, python3-matplotlib
- [ ] Verificarea extensiilor SIMD disponibile (`/proc/cpuinfo`)
- [ ] Structura proiectului (`src/common` , `src/scalar` , `src/sse` , `src/avx2` , `src/avx512` , `bench` , `scripts`)
- [ ] Versiunile scalare ale celor 3 algoritmi
- [ ] Harness-ul de masurare: timer, rulari repetate, mediana, validarea rezultatelor, export in `results.csv`
- [ ] Makefile cu flag-uri separate per backend (scalarul compilat cu `-fno-tree-vectorize`)

Laboratorul 4 - Primele variante vectorizate

De prezentat la laborator:

- [ ] Varianta scalara + harness functional, cu primele masuratori
- [ ] Produsul scalar vectorizat (SSE + AVX2), validat
- [ ] Convolutia 2D vectorizata (AVX2), validata

De facut pana la laboratorul 5:

- [ ] Analiza alinierii: buffer aliniat (`load`) vs nealiniat (`loadu`)
- [ ] Analiza dimensiunii vectorului: SSE (4 float) vs AVX2 (8 float)
- [ ] Inmultirea matricelor: naiv, cu B transpus, vectorizat AVX2
- [ ] Masuratori pe dimensiuni de date de la L1 pana la RAM

Laboratorul 5 - Optimizare si imbunatatirea pentru nota 10

De prezentat la laborator:

- [ ] Toti cei 3 algoritmi vectorizati si validati
- [ ] Analiza accesului la memorie la inmultirea matricelor
- [ ] Backend-ul AVX-512

De facut pana la laboratorul 6:

- [ ] Acumulatori multipli + loop unrolling (A1, A2)
- [ ] Cache blocking (tiling) la inmultirea matricelor, cu test pe mai multe dimensiuni de bloc
- [ ] Graficele finale: timp si speedup in functie de dimensiune, pentru fiecare algoritm si backend
- [ ] Raportul tehnic: arhitectura, implementare, teste, rezultate, interpretare

Laboratorul 6 - Predarea finala

- [ ] Demonstratie practica: compilare, rulare, validare, afisarea rezultatelor
- [ ] Prezentarea graficelor si a interpretarii lor
- [ ] Raportul tehnic final in repository
- [ ] Ultimul commit cu versiunea finala

---

5. Livrabile finale

Livrabil	Locatie in repository
Cod sursa (scalar, SSE, AVX2, AVX-512, optimizari)	`src/`
Harness de benchmark	`bench/`
Script pentru grafice	`scripts/plot.py`
Rezultate masuratori	`results/`
Roadmap	`ROADMAP.md`
Cadru conceptual	`CADRU_CONCEPTUAL.md`
Raport tehnic	`docs/RAPORT.md`

6. Criterii de evaluare urmarite

- Corectitudinea rezultatelor (comparatie cu varianta scalara)
- Timpii de executie si speedup-ul pentru fiecare algoritm
- Interpretarea rezultatelor (de ce speedup-ul variaza cu dimensiunea datelor)
- Beneficiul masurat al imbunatatirii pentru nota 10

7. Riscuri si solutii

Risc	Solutie
Compilatorul vectorizeaza automat varianta scalara si speedup-ul pare fals	Varianta scalara este compilata cu `-fno-tree-vectorize`; se verifica assembly-ul generat
Masuratori instabile sub WSL / Windows	Rulari repetate, raportarea medianei, fixarea procesului pe un nucleu
Diferente mici intre rezultatele float scalare si vectorizate	Validare cu toleranta relativa; ordinea adunarilor difera
Dimensiunea datelor nu se imparte exact la latimea vectorului	Tratarea separata a elementelor ramase (bucla de final)